

Assessment of Approaches for the Analysis of Refactoring Activity on Software Repositories

An Empirical Study

Matteo Orrú

Dept. of Electrical and Electronic Engineering
University of Cagliari
Piazza D'Armi 09123
Cagliari, Italy
matteo.orrú@diee.unica.it

Michele Marchesi

Dept. of Electrical and Electronic Engineering
University of Cagliari
Piazza D'Armi 09123
Cagliari, Italy
michele@diee.unica.it

ABSTRACT

Refactoring is the practice of changing source code without altering its external behavior. It is widely used since it is acknowledged to have a positive effect on software quality. However, different studies on the impact of Refactoring on software quality led to contrasting results. This might be due also to the fact that research works on Refactoring rely on different approaches to collect information about the refactoring activity on software repositories. Having a reliable approach is fundamental to draw solid conclusions. In this paper we empirically analyzed two of the most popular approaches to detect the refactoring activity on software repositories, finding that they return different outcomes. These results should be taken into account by researchers while designing the experimental settings of their studies.

CCS Concepts

•General and reference → Empirical studies; Measurement; Metrics; •Software and its engineering → Software configuration management and version control systems; Software version control; *Agile software development*; *Maintaining software*; Software evolution; Open source model;

Keywords

Refactoring, Mining Software Repositories, Developers Activity, Java Software Systems

1. INTRODUCTION

Refactoring aims at improving source code quality by performing changes that preserve software behavior. It was introduced by Opdyke in his PhD thesis [10] and later popularized in the famous book of Fowler [3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

XP '16 Workshops, May 24 2016, Edinburgh, Scotland UK

© 2016 ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2962695.2962717>

Refactoring is empirically acknowledged to be useful for improving software quality and many scholars focus their efforts to rigorously validate these claims. However, the research carried out so far yielded contrasting results. Some researchers found that Refactoring seems to negatively affect the quality of software systems [7]. In some cases, this is reflected in an increase of bug fixes after refactoring operations [6]. Other scholars found a positive correlation between Refactoring and quality [14]. Since the experimental settings are different it is difficult to compare the results.

One of the major difference regards both the *source* and the *method* used to retrieve information about the refactoring activity. Depending on the combination of sources and methods we can distinguish 4 different approaches:

1. **Mining the commit logs:** the source is represented by the commit logs stored in control version repositories (like *git* or *subversion*). Each commit is associated to the changes performed to the code during software development. This method is based on the research of some meaningful keywords (like “refactor” or “restructuring”) that might suggest that refactoring operations took place [15, 14].
2. **Analyzing the code history:** the source is represented by the source code belonging to different releases (not necessarily adjacent) of a software system. This method involves a syntactic analysis of the differences between the compared versions [19, 12].
3. **Using tools to log the refactoring activities:** this approach rely on the use of specific tools that record and keep track of the development activities. These tools are usually embedded in development environments, as it happens with plugins in Eclipse [9].
4. **Observing developers:** in this case the researchers are able to directly or indirectly observe the developers’ activities [9].

Each of these methods has its own advantages and drawbacks that might affect the results of a research in different ways. Some authors compared these different approaches highlighting the advantages and disadvantages of their adoption [8, 16]. Also, several authors addressed their efforts to improve the efficacy of these methods and to reduce their limitations [16].

Since the results of the research on Refactoring might be biased by the chosen approach it is mandatory to thoroughly understand if and to which extent these approaches differ and which kind of information they return. Moreover, depending on the goal of the specific study, it might be helpful to adopt manifold different approaches together [8]. Consequently, it is important to figure out weather and to which extent the approaches return the same information.

In this work we compared the first two of the mentioned approaches (“Mining the commit logs” and “Analyzing code history”) using a popular algorithm developed by Ratzinger et al. [15] (for the first approach) and the *state-of-the-art* tool for refactoring detection, RefFinder [5] (for the second one). We applied these two approaches to retrieve information about the refactoring activity that took place during the development of two popular software systems written in Java: Apache Ant¹ and Spring Framework².

The *goal* of this study is to empirically investigate if and to which extent the information on the refactoring activity returned using these two different approaches coincide. We found that the two approaches mostly return different outcomes, namely they do not detect the same refactoring operations, with some relevant differences in some cases. The results of the present work, although preliminary, might be of some help for researchers who are performing studies on refactoring. Moreover they could also be helpful to scholars and practitioners involved in the design of tools for refactoring automation.

The remainder of the present paper is structured as follows. In Section 2 we illustrated the recent works in the field. In Section 3 we described the dataset and the experimental settings while in Section 4 we illustrated the results. Next, in Section 5, we discussed the threats to validity. At the end, in Section 6, we drew the conclusions and outlined some ideas for the future works.

2. RELATED WORK

Refactoring is the activity of modifying the source code while preserving the external behavior [3, 10]. Since the publication of Fowler’s book, Refactoring has been a topic of increasing interest for both researchers and practitioners. In order to investigate the impact of Refactoring on software evolution it is mandatory to retrieve information about the refactoring activity that take place during development. From the software engineering research literature, as reported in Murphy-Hill et al. [8], we can presently distinguish the 4 approaches mentioned in the previous Section: mining the commit logs, analyzing the code history, using specific tools, usually plugins, to track the refactoring activities, and observing developers’ work [16, 8]. Each of them presents advantages and disadvantages.

In the present work we are focusing our attention on the first two of the cited approaches. Using the first one, Stroggylos and Spinellis found a relationship between refactoring and software quality [17]. In Parnin et al. [11] the authors rose some doubts on the usefulness of adopting the commit logs as a source to detect the refactoring activity, highlighting the incompleteness of the analyzed commits. Ratzinger [15, 14] analyzed software repositories in order to predict refactoring opportunities and to investigate the relationship

between Refactoring and software quality. Hindle et al. investigated the nature of large commits, collecting interesting information about refactoring related commits [4].

Several developers used the second approach to track the refactoring activity of developers. Weißgerber et al. [19] analyzed the code history taken from CVS. Xing et al. performed a study on Eclipse [20]. Many scholars developed tools for the automatic refactoring detection starting from the analysis of the source code. Some examples are: RefactoringCrawler [2], RefacLib [18], UMLDiff [20] and RefFinder [12]. To the best of the authors’ knowledge, RefFinder³ is presently the state-of-the-art tool for detecting the refactorings performed on software systems [12, 5]. It is based on a template-based reconstruction approach and it is able to detect 65 out of the over 70 refactorings in the original Fowler’s catalog [3], including both atomic and composite refactorings.

Some authors analyzed all these approaches, underlining their strengths and weaknesses [8]. In Soares et al. the authors compared some of these approaches, finding that the most reliable is the manual analysis. However, this approach is time consuming and it is highly influenced by the level of expertise of the researcher [16]. Although they use RefFinder in the second part of their work, they did not compare its outcome with that obtained by applying the first approach. Consequently, the comparison between the first two reported approaches remains, to the best of the authors’ knowledge, still unexplored. In the present work we relied on the algorithm proposed by Ratzinger [15] and on RefFinder [12]. We then compared the retrieved information to figure out weather and to which extent they coincide.

3. EXPERIMENTAL SETTINGS

3.1 Methodology

We adopted the Victor Basili’s *Goal-Question-Metric* approach[1]. According to this approach we set the **goal(s)**, we then formulated the **question(s)** and finally we designed the **metric(s)** to be used to answer the questions.

Goal: The goal is to investigate if and to which extent we can rely on the mentioned approaches to detect the refactoring activity on software repositories. These approaches, namely *mining of commit’s log* (first approach) and the *analysis of code history* (second approach), are two of the four proposed in the literature to accomplish this specific task. We can not ignore that experimental settings have a significant impact on the results of a research. Thus researchers should be aware that retrieving different information by using different approaches might in principle lead them to formulate different interpretations and draw wrong conclusions.

Question: Our questions aim at empirically evaluating if and to which extent the proposed approaches return the same or significantly different information. In the first case we can use the two approaches indifferently. In the second case we can conclude that adopting the first or the second approach might influence the interpretation and the conclusion drawn in a research work. Moreover, different methods put a emphasis on different aspects of software development. For example, commit log analysis assumes that developers

¹<http://ant.apache.org>

²<https://spring.io/>

³<https://sites.google.com/site/reffindertool/>

properly comment the changes performed during development. However, there could be many circumstances in which it may not occur. With this in mind, the researchers should carefully consider the approach that is best suited to their research goals. All that said, we formulated the following question:

RQ: Does *information* about the refactoring activity returned by the two methods coincide or differ significantly?

Metric: In order to answer to the research question, we used the following metrics. We firstly retrieved the number of commits associated to the refactoring activities using both the approaches. We will call the set of commits (*refactoring commits*) retrieved with the two approaches, respectively, *RC1* (with the first approach) e *RC2* (using the second approach). Then we found the commits that belong to both the first and the second set of commits (i.e. common commits *RC*). The higher is the number of commits that belong to this last set, the higher is the correspondence between the information returned using the first or the second approach.

3.2 Approaches

We performed the refactoring extraction using the following approaches:

- **Commit Log Analysis (Ratzinger):** we retrieved the repositories activities and stored all the information in a MySQL database. Then we queried the database looking at commits with specific keywords [15, 13]. We used a set of keywords already presented in Ratzinger [13], following the algorithm reported in [16]. The keywords are the following:

```
refactor, restruct, clean, not used, unused,
reformat, import, remove, removed, replace,
split, reorg, rename, move
```

We also discarded comments containing the “**needs refactoring**” keyword. In the following we refer to this approach as **Ratzinger**.

- **Code History Analysis (RefFinder):** After having retrieved the source code from each software repository we selected a number of releases. In our analysis we considered the releases pairs, namely the adjacent releases in the set of the selected ones. Then we used RefFinder to collect the refactorings performed during the development that took place between two releases of each release pair [3]. RefFinder adopt a template-reconstruction approach and works comparing two releases looking for differences: when it finds differences that fit with one of its templates, namely with its refactoring definitions in a formal language, the tool returns information on the specific detected refactoring, including the class where it was performed [5, 12]. Since we need to associate to each refactoring the corresponding commit, we used `git blame`, a feature of the git source control manager. This tool returns information on all the changes performed on a specific file, including author, commit hash, date and time. We linked this information with the results returned by RefFinder. In the following we refers to this approach as **RefFinder**.

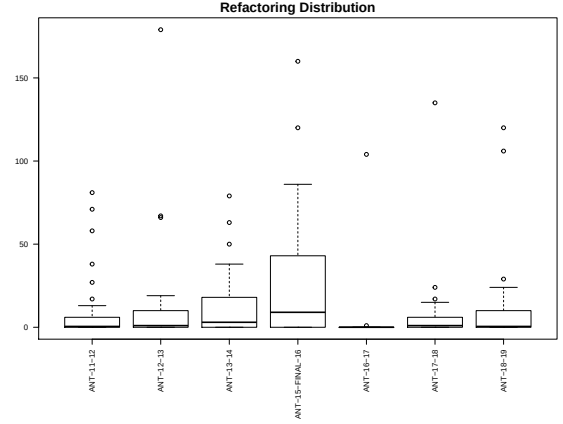


Figure 1: Distribution of refactorings for Apache Ant

3.3 Dataset

In this work we analyzed two popular software systems (Apache Ant and Spring Framework). The first one is a popular library and command-line tool for automating software building tasks. Being a system often studied in research on refactoring it has been chosen to enable comparisons of results. Spring Framework is a project of the popular framework for development of enterprise systems and applications. Both the systems are written in Java. We choose to study this system because it was studied by Ratzinger in his works on refactoring detection [15, 13]. We collected the entire history of both the systems from their git repositories. In the following Sections we detailed some of the salient features of the two datasets.

3.3.1 Ant

The development of Ant encompass a period of time ranging from 2000-01-13 to 2015-12-14 for a total of 16 years. 58 developers took part to the development during this time, and committed 15477 times.

In this preliminary analysis we focus mainly on the major releases, reported on Tab. 1 along with the corresponding release dates.

3.3.2 Spring Framework

We collected several releases of the version 3 of this system, whose features are enlisted in Tab. 2. The development of Spring embrace a period of time ranging from 2009-12-16 till 2011-12-22 for a about 2 years.

In this preliminary analysis we focus mainly on the releases that reports a significant number of refactorings, as reported on Tab. 2, according to the Ratzinger algorithm. In the same Table we highlighted the interested releases in bold, and we marked with a number of asterisks those releases that are not adjacent. This way we were able to collect a meaningful number of refactorings between the releases to enable our analysis.

In Tables 1 and 2 the releases are paired together (source and target releases) because RefFinder works by comparing two versions of the same system. The column “Ref. Commit Num.” reports the number of commits that are associated to refactorings according to the Ratzinger algorithm.

Apache Ant							
Source Name (Tag)	Source Date	Target Name (Tag)	Target Date	Days	Ref. Commit Num.	Commit Num.	
ANT_11	2000-07-18	ANT_12	2000-10-24	98.04	42	327	
ANT_12	2000-10-24	ANT_13	2001-03-01	128.17	88	629	
ANT_13	2001-03-01	ANT_14	2001-09-03	185.87	125	921	
ANT_15.FINAL	2002-07-09	ANT_160	2003-12-18	526.95	356	2739	
ANT_160	2003-12-18	ANT_170	2007-10-29	1411.2	494	4776	
ANT_170	2007-10-29	ANT_180	2010-02-02	826.3	102	1209	
ANT_180	2010-02-02	ANT_190	2013-03-06	1128.15	67	836	

Table 1: Analyzed releases of Apache Ant

Spring Framework							
Source Name (Tag)	Source Date	Target Name (Tag)	Target Date	Days	Ref. Commit Num.	Commit Num.	
v3.0.0.RELEASE (*)	2009-12-16	v3.0.1.RELEASE	2010-02-18	64.04	14	320	
v3.0.1.RELEASE	2010-02-18	v3.0.1.RELEASE.A	2010-02-19	0.74	0	5	
v3.0.1.RELEASE.A	2010-02-19	v3.0.1.RELEASE-A	2010-02-19	0	1	2	
v3.0.1.RELEASE-A	2010-02-19	v3.0.2.RELEASE (*)	2010-04-02	41.91	10	200	
v3.0.2.RELEASE (**)	2010-04-02	v3.0.3.RELEASE	2010-06-15	74.15	7	195	
v3.0.3.RELEASE	2010-06-15	v3.0.4.RELEASE (**)	2010-08-19	64.86	15	183	
v3.0.4.RELEASE(***)	2010-08-19	v3.0.5.RELEASE	2010-10-20	62.53	9	165	
v3.0.5.RELEASE	2010-10-20	v3.0.6.RELEASE (***)	2011-08-19	302.08	136	983	
v3.0.6.RELEASE	2011-08-19	v3.1.0.RELEASE	2011-12-13	116.68	54	535	

Table 2: Analyzed releases of Spring Framework (in bold). We marked with asterisks the not adjacent releases

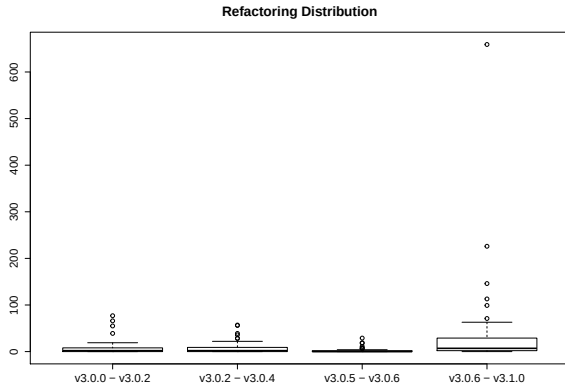


Figure 2: Distribution of refactorings for Spring Framework

4. RESULTS

Tables 1 and 2 report some basic features of the analyzed releases, including the total number of commits occurred between the two releases. Figures 1 and 2 show the respective statistical distribution of the number of refactorings. Each point represents the number of occurrences of a specific refactoring (i.e. “add parameter”, “rename method”, etc. as detected by RefFinder) of the most widespread refactorings of any systems or, in other words, the outliers. Figures 3 and 4 report the names of the outliers for respectively Ant and Spring. The common refactorings for the two systems are highlighted in bold. Tables 5 and 6 report all the refactorings detected by RefFinder along with their distribution (i.e. number of occurrences) for each paired releases.

The detailed analysis is reported, for each system, in the corresponding section in the following.

4.1 Apache Ant

As we can see from Tab. 1, we studied 8 releases of Apache

Ant, to which we applied both the mentioned approaches. There are significant differences in the number of detected refactorings of the different releases. Moreover, to different kinds of refactorings correspond very different frequencies. The median is constantly under 20, which means that in the vast majority of the releases, half of the refactorings are performed rather rarely, if compared to the third quartiles, maximum and outliers values of almost each release. Also it is worth to note that, as shown in Fig. 1, for the paired releases Ant 1.6 and Ant 1.7, the distribution of refactorings shows some peculiarities, suggesting that something unusual occurred during the development activities.

RefFinder found 34 different refactorings out of the 65 it can detect, whose names are reported in Tab. 5 along with their number of occurrences. As illustrated in Fig. 1, each distribution shows several outliers whose names are reported in Fig. 3. In Tab. 3 is reported the number of refactoring commits, namely commits that refer to the refactoring activities as detected using the Ratzinger approach (RC1). The refactoring commits are 42, 88, 125, 356, 494, 102 and 67 and amount to the 12%, 14%, 13.6%, 13%, 10.3%, 8.4% and 8% of all the commits, respectively. The same Table reports also the number of commits associated to the refactoring activity detected by RefFinder (RC2). These values are, for each release, respectively 186, 80, 130, 272, 490, 96 and 105 corresponding to the 56.9%, 12.7%, 14.1%, 9.9%, 10.3%, 7.9% and 12.6% of all the commits, respectively.

There are differences in magnitude but, with the exception of the 1st, 4th and 7th release pair, for which the difference in the number of commits are relevant, in general the number of commits retrieved with both the approaches is approximately the same. In one case we find that Ratzinger algorithm retrieve a significantly higher number of refactoring (“Ant 1.5 - Ant 1.6”) whereas in two cases (“Ant 1.1 - Ant 1.2” and “Ant 1.8 - Ant 1.9”) RefFinder detected a meaningfully higher number of refactoring.

The number of common commits is considerably low, being equal to the 7.3%, 1.4%, 1.6%, 1.3%, 1.4%, 0.7% and 0.8% of the total number of commits for the respective ver-

Apache Ant				
Source	Target	RC1 (Ratzinger)	RC2 (RefFinder)	RC (Common)
1.1	1.2	42	186	24
1.2	1.3	88	80	9
1.3	1.4	125	130	15
1.5	1.6	356	272	36
1.6	1.7	494	490	69
1.7	1.8	102	96	9
1.8	1.9	67	105	7

Table 3: Number of refactoring commits retrieved using the two approaches for Apache Ant. The fifth column reports the number of common commits

sion pairs and to the 12.9%, 11.2%, 11.5%, 13.2%, 14.1%, 9.4% and 6.7% of the refactored commits found with RefFinder and to the 57.1%, 10.2%, 12.0%, 10.1%, 14.0%, 8.8%, 10.4%, of those found with Ratzinger.

4.2 Spring Framework

Fig. 2 shows the distribution of the number of refactoring. If we consider all the releases we can point out that RefFinder detected 37 different refactorings whose names are reported in Tab. 6 along with their number of occurrences in each pair of releases. As it is shown in the Fig. 2, we can see that each distribution presents several outliers. The names of the outlier refactorings are reported in Fig. 4.

As reported in Tab. 4 the number of refactoring commits, namely commits that refer to the refactoring activities, are respectively, 25, 22, 145 and 54, which correspond to the 4.6%, 5.8%, 12.6% and 9.9% of all the commits, respectively. The same Table reports also the values for the commits associated to the refactoring activities detected by RefFinder. The numbers are, for each releases, respectively 63, 36, 13, 176, which correspond to the 12%, 9.5%, 1.1% and 32.9% of the all the commits, respectively.

Also the differences in magnitude are meaningful and show that commits associated to the refactoring activity detected by RefFinder are far more numerous than those detected using the Ratzinger algorithm in 4 cases out of 5. In many cases the difference is also significant, being the number of refactoring commits found by RefFinder two times the number of the corresponding number retrieved by the Ratzinger algorithm for the first three releases (first two paired releases) and more than three times in the case of the last two releases.

Interestingly enough, the only case in which the Ratzinger algorithm detected a higher number of refactoring, the difference is far higher than the other way around. Tab. 4 also shows the number of commits that are retrieved from both the approaches. The number is quite low, corresponding to, respectively, the 8.3%, 1.4% and 45.3% and 3.2%, 15.4%, 13.6% of the first, third and fourth pair for both the Ratzinger and the RefFinder case. With the exception of the third release of the Ratzinger case, the values are constantly under the 20%.

Some interesting considerations arise from the comparison of the reported results for the two systems. First and foremost, it is worth to note that the analyzed releases of the two systems have different characteristics. The average number of commits for Ant is 1633.86 whereas for Spring is 647 with a median of, respectively, 921 and 531. The number of commits is for Ant is 2.53 times higher than for Spring.

Spring Framework				
Source	Target	RC1 (Ratzinger)	RC2 (RefFinder)	RC (Common)
v3.0.0	v3.0.2	25	63	2
v3.0.2	v3.0.4	22	36	0
v3.0.4	v3.0.6	145	13	2
v3.0.6	v3.1.0	54	176	24

Table 4: Number of refactoring commits retrieved using the two approaches for Spring Framework. The fifth column reports the number of common commits

Also the number of refactored commits, namely those associated to refactoring operations according to the Ratzinger algorithm, are different: the mean number is 182 for Ant and 61 for Spring with median, respectively, 102 and 38.5. Thus the mean number of refactored commits are 2.98 times higher for Ant if compared to Spring.

We have on average 614.95 days between the releases for Ant, whereas for Spring the development days between the releases are 181.75, so Ant has 3.38 times the development days between the releases if compared to Spring. All that said it seems that there is approximately the same ratio between the total number of commits and the number of refactored commits, as well as for the number of days and the number of commits. It seems to suggest that, although the systems are quite different, the more commits we have, the more refactored commits we find. Among these commits the percentage of refactored commits is roughly the same for both the systems.

Furthermore, regarding the common commits, the results are congruent for both the systems. The number of common commits, is for both of them quite low, being around the 10% if we considered the fraction of refactored commits (retrieved with both the approaches) with episodic fluctuations and some peaks (one per system).

It is worth to note that there is a significant difference in the number of refactored commit retrieved with both the approaches. For Ant the number of refactored commits retrieved is similar for the paired releases most of the times, whereas for Spring we have the opposite behavior.

This lead us to conclude, at least with regards to our dataset, that the results returned by the two methods are quite different. Thus we are able to answer to the posed research question.

RQ: Does *information* about the refactoring activity returned by the two methods coincide or differ significantly?

A: The answer to this research question is **negative**.

Since the information returned by the two methods might affect the results of an experiment, this difference should be taken in great account while designing an experimental setting. This result, although preliminary, might pose a caveat on the available studies on refactoring detection (possibly those which adopted the studied approaches).

As suggested by Emerson et al. [8] a thorough analysis of the different approaches would be certainly valuable to improve the understanding of the effectiveness of the different research methods. In order to mitigate the effect of the reported differences a solution might be to combine the diverse approaches. Also selecting complimentary approaches,

“add parameter” “introduce null object” “move field” “remove parameter” “rename method”
 “replace method with method object” “**replace magic number with constant**” “remove control flag”
 “replace constructor with factory method” “extract method” “**introduce explaining variable**”

Figure 3: List of the most used refactorings (outliers) of Apache Ant. In bold the outliers in both the systems

“add parameter” “consolidate duplicate cond fragments” “**introduce explaining variable**”
 “**introduce null object**” “move field” “move method” “**remove parameter**” “**rename method**”
 “**replace magic number with constant**”

Figure 4: List of the most used refactorings (outliers) of Spring Framework. In bold the outliers in both the systems

or approaches that are less prone to bias the result of the research, might be appropriate in some cases.

5. THREATS TO VALIDITY

This work suffers from some threats to validity, that should be considered in case of replication study. In this Section we present them, following the typical division in threats to the internal, external and construct validity.

Internal We found empirical evidence that the studied approaches to detect the refactoring activity on software repositories return contrasting information. A possible interpretation might be that developers do not comment properly the refactoring activities. Also the current software development practices might have an impact on what results from a commit’s comment. For example, for simple refactorings (i.e. “pull-up method”, etc.) popular IDE provide automatic refactoring features. The use of these features may in principle induce developer not to pay enough attention to the specific changes because they occur during the “flow” of programming.

External In this work we did not perform an analysis on all the refactorings, but we focused on the outliers in order to have a better statistical base. Thus, some of the results might not apply to the discarded refactorings. Extending the analysis is needed to better validate our results. Moreover, we analyzed only two open source software systems. We focus on open source software in order to exploit the opportunity to parse the source code and access to their versioning systems. However, different dynamics that might take place in proprietary software development environments might have an impact on the refactoring activities. This might affect the results if the presented analysis is replicated on such systems. Also there might be some differences depending on the programming language.

Construct We used RefFinder to retrieve the refactorings.

It is the *state-of-the-art* software for this task, with the widest coverage in this moment. However, it retrieves a subset of the refactoring on the original Fowler’s catalog. There is not a clear evidence that some of this “not detected” refactorings are not often used in the analyzed software systems. Since we can not be sure that some of these refactorings could not be included into the outliers there is some likelihood that results might change while studying these “not covered” refactorings.

6. CONCLUSION

In this work we presented the results of an empirical analysis aimed at comparing the information returned by two popular approaches for detecting the refactoring activity on software repositories. This research was conducted on several releases of two popular software systems: Apache Ant and Spring Framework. We retrieve data about the refactoring activities by mining their git repository. Then we selected the commits related to the refactoring activities using the Ratzinger algorithm. We also analyzed the source code using RefFinder and then, by leveraging the git blame functionality, we found the commits associated to the retrieved refactorings. Then we compared the results, namely we count the number of commits retrieved using each approach and the number of common commits.

We found that the commits associated to the refactoring activities retrieved using these two methods are different, meaning that the two approaches retrieve different information. Since these approaches might have an impact on the results this finding should be taken into account while designing the experimental settings.

As future work we are planning to extend the analysis to a larger set of software systems, including systems written in different languages. It is also our intention to extend the analysis to all the refactorings and not just to the most used ones. Moreover, we are planning to deepen our analysis by performing an investigation on specific refactorings and to improve the statistical analysis to address the reported threats to validity.

7. REFERENCES

- [1] V. R. Basili. Software modeling and measurement: The goal/question/metric paradigm. Technical report, College Park, MD, USA, 1992.
- [2] D. Dig, C. Comertoglu, D. Marinov, and R. Johnson. Automated detection of refactorings in evolving components. In *Proceedings of the 20th European Conference on Object-Oriented Programming*, ECOOP’06, pages 404–428, Berlin, Heidelberg, 2006. Springer-Verlag.
- [3] M. Fowler. *Refactoring: improving the design of existing code*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1999.
- [4] A. Hindle, D. German, and R. Holt. What do large commits tell us? a taxonomical study of large commits. pages 99–108, 2008.
- [5] Kim, M. Gee, A. Loh, and N. Rachatasumrit. Ref-finder: A refactoring reconstruction tool based on logic query templates. In *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on*

Apache Ant							
Refactoring	1.1 - 1.2	1.2 - 1.3	1.3 - 1.4	1.5 - 1.6	1.6 - 1.7	1.7 - 1.8	1.8 - 1.9
add parameter	81	67	79	83	0	3	120
change uni to bi	0	0	1	0	0	0	0
consolidate cond expression	2	5	4	15	0	6	3
consolidate duplicate cond fragments	9	9	24	47	0	12	6
decompose conditional	0	0	0	1	0	0	0
extract interface	3	0	4	6	0	1	0
extract method	3	19	18	43	0	17	13
extract superclass	0	0	2	9	0	1	0
hide delegate	0	1	0	2	0	0	0
inline method	0	3	4	3	0	3	1
inline temp	3	2	9	35	0	14	10
introduce assertion	0	0	0	1	0	1	5
introduce explaining variable	3	14	20	39	0	24	19
introduce local extension	0	0	7	10	0	0	2
introduce null object	17	12	20	26	0	0	0
move field	27	13	18	120	0	5	16
move method	13	10	19	84	0	9	11
pull up field	0	1	5	46	0	0	0
pull up method	0	1	3	18	0	0	0
remove assignment to parameters	6	2	5	35	0	15	8
remove control flag	4	3	3	9	104	11	8
remove parameter	71	66	63	63	0	3	106
remove setting method	0	0	0	0	0	0	1
rename method	58	179	38	160	0	7	24
replace constructor with factory method	0	0	0	0	1	0	0
replace data with object	1	0	2	0	0	1	2
replace error code with exception	0	0	0	1	0	0	0
replace exception with test	0	0	2	0	0	1	0
replace magic number with constant	10	15	50	86	0	135	29
replace method with method object	38	15	22	48	0	6	14
replace nested cond guard clauses	1	0	3	10	0	4	0
replace temp with query	0	2	0	0	0	0	0
self encapsulate field	0	1	0	0	0	0	0
separate query from modifier	1	0	0	0	0	0	0

Table 5: Number of refactorings found by RefFinder for Apache Ant

Spring Framework				
Refactoring	v3.0.0 - v3.0.2	v3.0.2 - v3.0.4	v3.0.5 - v3.0.6	v3.0.6 - v3.1.0
add parameter	138	57	18	146
change bi to uni	2	0	0	0
change uni to bi	2	1	0	0
collapse hierarchy	0	0	0	2
consolidate cond expression	12	5	1	8
consolidate duplicate cond fragments	29	29	7	29
extract hierarchy	1	1	0	1
extract interface	6	4	0	7
extract method	21	22	2	35
extract subclass	0	0	0	3
extract superclass	0	0	0	1
inline method	3	7	2	10
inline temp	6	9	2	19
introduce assertion	31	9	4	13
introduce explaining variable	26	28	6	48
introduce local extension	0	0	0	9
introduce null object	5	2	0	659
move field	11	2	8	63
move method	9	22	29	99
parameterize method	0	0	0	7
preserve whole object	0	0	0	1
push down method	0	0	0	2
remove assignment to parameters	10	3	3	15
remove control flag	0	4	0	0
remove parameter	119	56	19	113
remove setting method	2	0	0	0
rename method	52	39	0	226
replace conditional with polymorphism	1	1	0	2
replace constructor with factory method	1	0	0	8
replace data with object	0	0	0	2
replace exception with test	2	0	2	2
replace magic number with constant	108	36	11	71
replace method with method object	12	5	0	52
replace nested cond guard clauses	6	4	2	3
replace subclass with field	0	0	0	1
replace temp with query	0	1	0	3
separate query from modifier	3	16	0	3

Table 6: Number of refactorings found by RefFinder for Spring Framework

- Foundations of Software Engineering*, FSE '10, pages 371–372, New York, NY, USA, 2010. ACM.
- [6] M. Kim, D. Cai, and S. Kim. An empirical investigation into the role of api-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering, ICSE 2011, Waikiki, Honolulu , HI, USA, May 21-28, 2011*, pages 151–160, 2011.
 - [7] M. Kim, T. Zimmermann, and N. Nagappan. An empirical study of refactoring challenges and benefits at microsoft. *IEEE Trans. Software Eng.*, 40(7):633–649, 2014.
 - [8] E. Murphy-Hill, A. Black, D. Dig, and C. Parnin. Gathering refactoring data: A comparison of four methods. 2008.
 - [9] E. Murphy-Hill, C. Parnin, and A. Black. How we refactor, and how we know it. *IEEE Transactions on Software Engineering*, 38(1):5–18, 2012.
 - [10] W. F. Opdyke. Refactoring object-oriented frameworks. Technical report, 1992.
 - [11] C. Parnin and C. Görg. Improving change descriptions with change contexts. In *Proceedings of the 2008 International Working Conference on Mining Software Repositories*, MSR '08, pages 51–60, New York, NY, USA, 2008. ACM.
 - [12] K. Prete, N. Rachatasumrit, N. Sudan, and M. Kim. Template-based reconstruction of complex refactorings. In *Proceedings of the 2010 IEEE International Conference on Software Maintenance, ICSM '10*, pages 1–10, Washington, DC, USA, 2010. IEEE Computer Society.
 - [13] J. Ratzinger. *sPACE: Software Project Assessment in the Course of Evolution*. PhD thesis, 2007.
 - [14] J. Ratzinger, T. Sigmund, and H. Gall. On the relation of refactoring and software defects. pages 35–38, 2008.
 - [15] J. Ratzinger, T. Sigmund, P. Vorburger, and H. Gall. Mining software evolution to predict refactoring. pages 354–363, 2007.
 - [16] G. Soares, R. Gheyi, E. Murphy-Hill, and B. Johnson. Comparing approaches to analyze refactoring activity on software repositories. *Journal of Systems and Software*, 86(4):1006–1022, 2013.
 - [17] K. Stroggylos and D. Spinellis. Refactoring—does it improve software quality? In *Proceedings of the 5th International Workshop on Software Quality, WoSQ '07*, pages 10–, Washington, DC, USA, 2007. IEEE Computer Society.
 - [18] K. Taneja, D. Dig, and T. Xie. Automated detection of api refactorings in libraries. In *Proceedings of the Twenty-second IEEE/ACM International Conference on Automated Software Engineering, ASE '07*, pages 377–380, New York, NY, USA, 2007. ACM.
 - [19] P. Weißgerber and S. Diehl. Identifying refactorings from source-code changes. pages 231–240, 2006.
 - [20] Z. Xing and E. Stroulia. Refactoring practice: How it is and how it should be supported - an eclipse case study. In *Proceedings of the 22Nd IEEE International Conference on Software Maintenance, ICSM '06*, pages 458–468, Washington, DC, USA, 2006. IEEE Computer Society.